四足机器人关节位置流
上位机 ⇄ 下位机（本机）软件通讯协议与接口 API

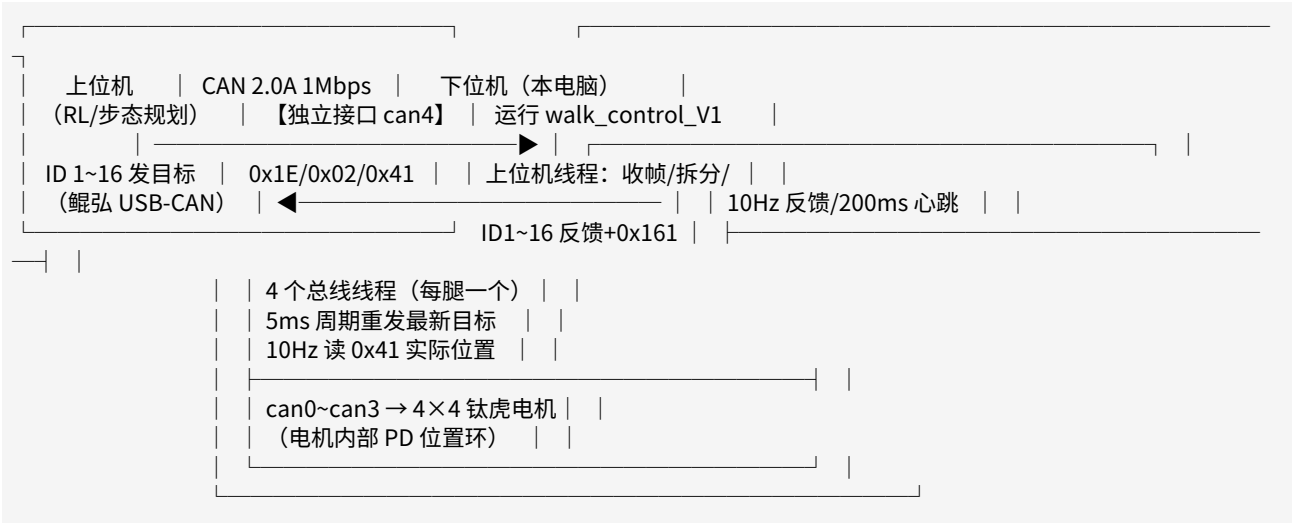
项目：TaiHu_motor_control / walk_control_V1

协议版本：V2.0 | 物理层：CAN 2.0A（标准帧，1 Mbps） | 通讯频率：10 Hz

本文档描述上位机（强化学习/轨迹规划层，**RK3588** ARM 嵌入式设备，aarch64，C/C++ + Linux SocketCAN）与下位机（本机，执行 walk_control_V1 程序）之间的 CAN 通讯协议。上位机将 16 个关节的预期位置以帧 ID 1~16 发送；下位机负责拆分——ID 1~4 → can0 电机 1~4，ID 5~8 → can1 电机 1~4，以此类推——经鲲弘 USB-CAN 下发到 4 条电机总线。帧结构与钛虎（TaiHu）电机写命令帧**同构**（DLC=5，[功能码][int32 小端]），上位机侧收发代码可直接复用控制真电机的写法。

第 7 章为上位机实现者必读：给出单帧字节布局、一个完整 10Hz 周期的 16 帧实际字节表、可直接编译的 C++ 完整骨架，以及 12 项出错排查对照表。

1. 系统架构与角色划分



职责划分：

层级	设备/程序	职责
上位机	另一台电脑（RL 策略/步态规划，鲲弘 USB-CAN 卡）	输出 16 个关节的预期位置，10Hz 以 ID 1~16 发送；监护心跳；结束时发 0x02 退出流模式
下位机	本电脑 + walk_control_V1	收上位机帧并按 ID 拆分到 4 条电机总线（ID n → 腿 (n−1)/4+1、电机 (n−1)%4+1）；关节端→电机端换算；5ms 周期重发最新目标；软件限位保护；10Hz 读实际位置并反馈上位机；心跳

物理连接：上位机 CAN 卡的 CAN_H/CAN_L 接入本机**独立**的一条 CAN 总线（对应接口如 can4/can5，程序启动时输入接口名）。**严禁**与电机总线 can0~can3 共用——否则上位机发的 ID 1~4 会被腿 1 的真电机直接当作位置命令执行，且 ID 5~16 在电机总线上无接收者（程序启动时会主动拒绝共用）。总线两端各并联 120Ω 终端电阻，波特率 1 Mbps（`sudo ./build/kcanctl up can4 1000000`）。

总线负载估算：上位机链路 16 帧 × 10Hz + 反馈 16 帧 × 10Hz + 心跳 5Hz ≈ 325 帧/s，约占 1Mbps 容量的 4%。每条电机总线 4 电机 × 200Hz 写帧 = 800 帧/s ≈ 9%，均无冲突风险。

2. 协议总览：16 关节 ID 设计

核心思想：上位机把 4 腿 × 4 关节共 16 个关节视为 16 个「虚拟电机」，帧 ID 1~16 一一对应。帧结构与钛虎电机写命令帧**完全同构**（DLC=5，[功能码 1B][int32 小端 4B]），只是把 int32 的语义从「编码器 cnt」重定义为「关节端角度(rad) × 10000」，把帧 ID 从「电机 ID」重定义为「全局关节 ID」。上位机控制真电机与控制本下位机使用同一套收发代码。

ID 映射：ID = (腿号−1)×4 + 关节号；反过来 腿号 = (ID−1)/4+1，关节号 = (ID−1)%4+1。即 ID 1~4 → 腿 1（can0）电机 1~4，ID 5~8 → 腿 2（can1）电机 1~4，ID 9~12 → 腿 3（can2），ID 13~16 → 腿 4（can3）。

方向	帧 ID	DLC	功能码	含义
上位机→下位机	1~16	5	0x1E	单关节目标位置帧：int32 = 关节端 rad × 10000（§3.1），10Hz 周期
上位机→下位机	1~16（任一）	1	0x02	退出流模式：电机保持当前目标，不抱闸
上位机→下位机	1~16	1	0x41	状态请求：下位机立即回同 ID 反馈帧（§3.3）
下位机→上位机	1~16	5	0x41	状态反馈：int32 = 该关节实际位置 rad × 10000，10Hz 周期发送（电机离线时发最近缓存值）；收到请求帧亦立即回复
下位机→上位机	0x161	1	—	心跳，200ms 周期：bit0=流模式中，bit1=最近1s有新样点

全部为 CAN 2.0A 标准帧（11 位 ID），数据字段多字节一律**小端序**（低字节在前，与钛虎电机协议一致）。

3. 帧格式详解

3.1 目标位置帧（0x1E，上位机→下位机）

一帧携带**一个关节**的目标角，16 帧组成一个完整样点，10Hz
周期发送。收到首帧即进入流模式；之后每个样点收到即生效（覆盖式，始终追踪最新目标）。

字节	0	1~4
内容	0x1E	角度（int32 小端，字节 1 为最低位）
类型	uint8	int32 LE
单位	功能码	关节端角度(rad) × 10000

角度编码：**定点 1 LSB = 0.0001 rad ≈ 0.0057°**，int32 有符号小端，范围远超关节行程，无饱和风险。例：J2 = +0.7914 rad（≈45.34°）→ 7914 = 0x00001EEA → 字节序 EA 1E 00 00。负角按补码：−0.5236 rad（−30°）→ −5236 = 0xFFFFEB7C → 7C EB FF FF。

语义：角度为关节端**绝对角**（弧度），正值 = 逆时针（ccw）。下位机内部自动完成关节端 → 电机端换算（J2/J3 锥齿轮 ×2、减速比、方向符号）并转为编码器 cnt 下发，上位机无需关心。

3.2 退出流模式帧（0x02，上位机→下位机）

DLC=1，数据 [0x02]，发到 ID 1~16 中任意一个均可。收到后下位机退出流模式，各电机**保持当前**目标位置（电机内部位置环维持，不抱闸）。用于正常结束轨迹回放。

3.3 状态请求与反馈帧（0x41）

上位机发 [0x41]（DLC=1，ID=1~16），下位机立即在同 ID 回 DLC=5 反馈帧；此外下位机也以 10Hz
周期主动发送全部 16 个 ID 的反馈帧（上位机可只收不请求）：

字节	0	1~4
内容	0x41	角度（int32 小端，字节 1 为最低位）
类型	uint8	int32 LE
单位	功能码	关节端实际角(rad) × 10000

反馈内容为该关节的**实际位置**（下位机以 10Hz 从电机 0x41
帧读取并缓存；电机离线时发最近缓存值，上位机可结合心跳 bit1 判断在线状态）。

3.4 心跳帧（下位机→上位机）

字段	帧 ID	DLC	数据
值	0x161	1	状态位：bit0=流模式中；bit1=最近1s有新样点

200ms 固定周期发送。上位机超过 600ms 未收到心跳 → 判定下位机失联，应停止轨迹规划并进入安全策略。

3.5 使能（解抱闸）语义 —— 上位机必须知道的前置条件

钛虎电机**上电默认去使能（抱闸锁死）**。在协议中**没有独立的「使能」命令**：去使能 = 功能码 0x02（本机用 estop/disarm 触发），而**使能 = 解抱闸 = 向电机写一条「电流 = 0」命令（功能码 0x1C，int32 = 0）**——电流模式下零转矩即自由，刹车松开。

这意味着：**若电机未使能，上位机发的 0x1E 位置帧会被正常接收、正常通过限位检查、正常写入目标缓存，但电机不会动**（抱闸锁着）。现场曾出现的「上位机发了轨迹、关节角一直是 0」即此原因，与帧格式无关。

本机 walk_control_V1 已内置使能：启动时对每条启用的腿自动执行「读实测位置 → 清错误位（0x0B）→ 逐电机写 0x1C=0（使能+零转矩，解抱闸）→
立即以刚读到的实测位置为目标切回位置模式」，因此**上位机无需发送任何使能帧**，直接进入 10Hz 位置流即可。

状态	触发方式	电机行为	上位机 0x1E 是否执行
已使能（解抱闸）	下位机启动时自动；键盘 arm / resume	电流 0 自由 →
立即被位置环接管，保持或跟随目标	**执行**

状态	触发方式	电机行为	上位机 0x1E 是否执行
去使能（抱闸）	键盘 estop / disarm；程序未使能	刹车锁死，位置环输出被机械阻断	帧被接收但**不产生运动**

⚠ 心跳帧只反映「流模式/样点在线」，**不反映使能状态**。上位机若需确认电机确实在跟随，请比较下发目标与 0x41 反馈实际角：连续若干周期反馈角不追随目标 ⇒ 电机可能抱闸或离线，此时应由现场人员在下位机终端执行 arm，而不是由上位机反复重发帧。

4. 10Hz 时序与执行语义

```
上位机时刻  0ms     100ms   200ms   300ms  ......
ID1~16帧  ──▶     ──▶     ──▶     ──▶    10Hz 稳定周期
          样点k   样点k+1  样点k+2  样点k+3  （16帧≈1.5ms发完）

下位机上位机线程（收到即写入各关节原子目标缓存）
          ↓store  ↓store  ↓store
下位机总线线程×4（5ms 周期，每线程独占一条 can）
          ──────────────────────────────────────▶ 持续重发最新目标
          重发q_k 20次 → 重发q_k+1 20次 → ...

真电机（钛虎，KP=4000 KD=60 位置环，PD 在电机固件内执行）
          平滑追踪 10Hz 阶梯目标（样点间过渡由电机 PD 完成）
```

执行策略（**覆盖式最新样点**，非队列回放）：

① 样点不排队——总线线程始终取「最新」目标，5ms 周期重发 0x1E 写帧（纯写不等回复，非阻塞）；上位机 10Hz 更新目标、电机 100ms
内跟上，天然无累积延迟。这是实时遥操/步态流的正确语义（丢帧只影响平滑度，不产生滞后堆积）。

② 样点间过渡由电机固件内位置环（KP=4000/KD=60）完成，下位机**不做软件插值**，控制延迟最小。

③ 上位机应保证样点序列本身平滑（相邻样点角变化 ≤ 0.035 rad（2°）/帧），10Hz 阶梯目标即被电机 PD 平滑追踪。

典型周期分配：上位机侧规划+发送 16 帧 ≤ 2ms；下位机上位机线程写原子缓存 <0.1ms；每条总线 5ms 周期下发 4 电机 × 0x1E 写帧约 0.5ms 总线占用；10Hz 的 0x41 读回每电机一次往返约 1ms。

5. 角度换算与限位

协议角度一律为**关节端**绝对角（弧度）。下位机内部换算：

```
电机输出端角 = 关节角(rad) × kMotorRadPerJointRad × kJointDirSign
编码器 cnt = 电机输出端角 / 2π × 262144 × 减速比（由 JointModule 完成）
 J1（Y 直驱）  ：×1   J2（P 锥齿轮1:2）：×2
 J3（P 锥齿轮1:2）：×2   J4（Y 直驱）  ：×1
 方向符号：kJointDirSign = {+1, +1, +1, +1}（正角=逆时针）
```

软件限位（判定一律按**关节端**进行；越限目标**逐关节丢弃**并计数——同一样点内其余关节不受影响；电机保持上一目标）。电机端等效范围 = 关节端限位 × kMotorRadPerJointRad（J2/J3 锥齿轮 1:2 为 ×2）：

关节	构型	电机 CAN ID	限位（关节端）	电机端等效	关节端总减速比
J1	Y 直驱	1	±90°	±90°	101
J2	P 锥齿轮 1:2	2	±120°	±240°	81×2 = 162
J3	P 锥齿轮 1:2	3	−30° ~ +180°	−60° ~ +360°	81×2 = 162
J4	Y 直驱	4	±90°	±90°	101

方向符号：kJointDirSign = {+1, +1, +1, +1}，基准为 taihu_singleleg_step_xCAN（现场已验证方向正确）——该 demo 按电机输出端下发关键帧「逆时针为正」，其 {0, 180, 180, 0} 中 ID2/ID3 同为 +180°，说明两个 P 型锥齿轮在电机端同向，故关节端与电机端同号。协议角度正方向 = 逆时针（ccw）。

减速比说明：减速比只参与速度换算；位置换算为 radToCnt/cntToRad（262144 cnt = 2π，输出端），不含减速比。

6. 异常处理与安全保护

场景	检测方	判定条件	动作
轨迹流断流	下位机	流模式中 1s 无新样点	自动退出流模式，电机保持当前目标（不抱闸），打印告警
上位机失联	上位机	600ms 无 0x161 心跳	上位机停止规划，走安全策略（如发 0x02 后重启）
目标越限	下位机	某关节角超限位表	仅丢弃该关节的目标（电机保持），stat 命令可查丢弃计数
电机离线	下位机	0x41 读取连续 3 次超时	该关节读操作退避 1s（不阻塞 5ms 控制环），stat 显示 ×
下位机急停	人工	键盘 estop	全部电机发 0x02 抱闸 + 退出流模式 + **锁定**：拒绝一切新目标（含上位机 0x1E 与 move/goto），直到键盘 resume 解除
程序退出	人工	键盘 quit	全部电机抱闸后关闭总线
move/goto 冲突	下位机	流模式中收到键盘 move/goto	提示暂缓（上位机流优先），不执行轨迹命令

7. 真实上位机如何下发位置指令（字节级实操）

本章是给**真实上位机**（**RK3588** 嵌入式板卡，aarch64，C/C++，Linux SocketCAN）的实现手册：从单个关节的字节拼装，到一个完整 10Hz 周期的 16 帧，再到可直接编译运行的完整程序骨架。读完本章即可独立实现下发端，无需阅读本仓库代码。

7.1 一帧的完整字节布局

目标位置帧永远是**标准帧（11 位 ID）、DLC = 5**。**不要**用扩展帧（下位机明确丢弃 29 位 ID 帧），**不要**发远程帧，**不要**改用CAN-FD 的 BRS/长帧——本协议是经典 CAN 2.0A。

偏移	长度	字段	取值 / 说明
帧头	—	CAN ID	1~16（11 位标准帧）。**不是**电机 ID，是全局关节 ID = (腿号−1)×4 + 关节号
帧头	—	DLC	固定 5（数据 5 字节）；命令帧/反馈帧都是 5
data[0]	1 B	功能码	0x1E = 设定目标位置
data[1]	1 B	int32 最低字节	value & 0xFF
data[2]	1 B	int32 次低字节	(value >> 8) & 0xFF
data[3]	1 B	int32 次高字节	(value >> 16) & 0xFF
data[4]	1 B	int32 最高字节	(value >> 24) & 0xFF
—	—	value	int32 有符号，小端；= 关节端角度(rad) × 10000，四舍五入

编码步骤（三步，无歧义）：

```
① 定点化：value = lround(joint_rad * 10000.0);  // 1 LSB = 1e-4 rad ≈ 0.0057°
② 拆字节：低字节放 data[1]，依次到 data[4]（小端！最容易错的一步）
③ 发帧 ：can_id = id（1~16），len = 5，标准帧，data[0] = 0x1E
```

示例 A（正角）：腿 1 的 J2 目标 +1.2168 rad（≈+69.72°）。value = 12168 = 0x00002F88，小端字节序 88 2F 00 00：

```
ID = 0x002 (腿1 J2 = (1-1)*4 + 2 = 2)，DLC = 5
data = 1E  88  2F  00  00
       ▲   └──────┬──────┘
       │      int32 = 12168 (小端)
       └ 功能码 0x1E
```

示例 B（负角）：腿 2 的 J1 目标 −0.5236 rad（−30°）。value = −5236，int32 补码 = 0xFFFFEB7C，小端字节序 7C EB FF FF：

```
ID = 0x005 (腿2 J1 = (2-1)*4 + 1 = 5)，DLC = 5
data = 1E  7C  EB  FF  FF
```

```
        └──────────────┬──────────────┘
        int32 = -5236（补码，小端）
※ 负值必须走补码。误用「绝对值 + 单独符号位」会导致反向飞车。
```

示例 C（零位）：目标 0 rad → value = 0 → data = 1E 00 00 00 00。注意「发 0」与「不发」是两回事：发 0 是命令该关节回到零位，不发则保留上一次目标。

验证一个字节都没错的土办法：把 data 用 cansend 手发一次，若关节转动角度与你的计算一致，说明整条链路（含小端序）正确：

```
# 腿1 J2 → +69.72°，即 value=12168：
cansend can4 002#1E882F0000
# 腿2 J1 → -30°，即 value=-5236：
cansend can4 005#1E7CEBFFFF
# 参考：几何上 J2 转 +69.72° 时关节端位移应肉眼可见（约 1/5 圈）
```

7.2 一个 10Hz 周期的 16 帧（含未映射关节）

上位机把 4 腿 × 4 关节共 16 个值打包，**每个周期发 16 帧**（ID 1~16 各一帧），帧间隔无要求（1Mbps 下 5 字节帧约 130μs 上线，16 帧全部发完 < 2.5ms，远小于 100ms 周期）。

7.2.1 以仓库 step_make.xlsx 的真实样点为例

该表只有 **L1_J2 / L1_J3 / L2_J2 / L2_J3** 四列有数据（峰值 +1.221730 rad = +70.00°），其余 12 个关节整列恒为 0。取 **t = 4.6 s** 的峰值样点，16 帧逐字节如下：

帧 ID	腿/关节	目标 rad	value(int32)	data[0..4] 十六进制
0x001	腿1 J1	0.000000	0	1E 00 00 00 00
0x002	腿1 J2	+1.216844	12168	**1E 88 2F 00 00**
0x003	腿1 J3	+1.216844	12168	**1E 88 2F 00 00**
0x004	腿1 J4	0.000000	0	1E 00 00 00 00
0x005	腿2 J1	0.000000	0	1E 00 00 00 00
0x006	腿2 J2	+0.737763	7378	**1E D2 1C 00 00**
0x007	腿2 J3	+0.737763	7378	**1E D2 1C 00 00**
0x008	腿2 J4	0.000000	0	1E 00 00 00 00
0x009~0x00C	腿3 J1~J4	0.000000	0	1E 00 00 00 00 ×4
0x00D~0x010	腿4 J1~J4	0.000000	0	1E 00 00 00 00 ×4

峰值 1.216844 rad → ×10000 = 12168.44 → 四舍五入 **12168**（0x2F88，小端 88 2F 00 00）；0.737763 rad → 7378（0x1CD2，小端 D2 1C 00 00）。

⚠ 未接线的关节也会发 0，而下位机会把它当作「回零」命令。若某个关节当前不在零位且不希望它动，有两个选择：（a）该 ID 本周期**不发**（保留上次目标）；（b）把它填成**当前实测角**（从 0x41 反馈读）而不是 0。仓库的 Python 示例脚本用 --only-mapped 只发有数据的列，正是方案（a）。

7.2.2 用 candump 观察一个周期的真实抓包

下位机终端另开窗口执行 `candump can4`，应看到 10Hz 的 16 帧成组出现（下面截取 t=4.6s 那一组；下位机回传的 ID 1~16 反馈帧与 0x161 心跳也会出现在同一窗口，反馈帧 data[0] 为 0x41，可据此区分方向）：

```
# ── 上位机 → 下位机（16 帧一组，data[0]=1E）──
 can4  001  [5]  1E 00 00 00 00
 can4  002  [5]  1E 88 2F 00 00    ← 腿1 J2 +1.2168 rad
 can4  003  [5]  1E 88 2F 00 00    ← 腿1 J3 +1.2168 rad
 can4  004  [5]  1E 00 00 00 00
 can4  005  [5]  1E 00 00 00 00
 can4  006  [5]  1E D2 1C 00 00    ← 腿2 J2 +0.7378 rad
```

```
can4  007   [5]  1E D2 1C 00 00     ← 腿2 J3 +0.7378 rad
can4  008   [5]  1E 00 00 00 00
can4  009 ~ 010  [5]  1E 00 00 00 00    （腿3 四关节）
can4  00D ~ 010  [5]  1E 00 00 00 00    （腿4 四关节）
—— 下位机 → 上位机（data[0]=41 反馈，10Hz；161 心跳 5Hz）——
can4  002   [5]  41 xx xx xx xx     ← 实际角，可用来核对追踪情况
can4  161   [1]  03              ← bit0=流模式, bit1=样点在线
```

帧速率核对：`candump can4 | ts` 观察，上位机侧应为 160 帧/s（16 帧 × 10Hz）；下位机侧反馈也是 160 帧/s，加心跳 5Hz。若上位机侧只有 4 帧/周期，说明只发了有数据的关节（方案 a）。

7.3 RK3588 上位机完整 C++ 骨架（Linux SocketCAN）

以下代码可直接在 **RK3588**（aarch64）上本机编译（`g++ -O2 -std=c++17`，也可用交叉工具链`aarch64-linux-gnu-g++`），只依赖 Linux 内核自带的 SocketCAN，**不需要**任何厂商 SDK，也**不需要**装 python-can / can-utils。把 `ifname` 换成你实际接到下位机的接口名（真实硬件为 **can4**）。

```
// host_stream.cpp —— 上位机位置流下发骨架
// 编译：g++ -O2 -std=c++17 -o host_stream host_stream.cpp
// 运行：./host_stream can4        (真实硬件)
//      ./host_stream vcan0       (本机自测)
#include <linux/can.h>
#include <linux/can/raw.h>
#include <net/if.h>
#include <sys/ioctl.h>
#include <sys/socket.h>
#include <unistd.h>
#include <cmath>
#include <cstdint>
#include <cstdio>
#include <cstring>
#include <chrono>
#include <thread>

static constexpr int     kJoints      = 16;   // 4 腿 × 4 关节
static constexpr double  kAngleScale  = 10000.0; // 1 LSB = 1e-4 rad
static constexpr uint32_t kHeartbeatId  = 0x161; // 下位机心跳
static constexpr uint8_t  kCmdSetPos    = 0x1E;  // 目标位置
static constexpr uint8_t  kCmdStop      = 0x02;  // 退出流模式
static constexpr uint8_t  kCmdGetStatus = 0x41;  // 状态请求/反馈
static constexpr int     kHeartbeatMs  = 600;   // 判失联门限

/// 打开 SocketCAN 接口并 bind（等价于 ip link set up + bind）。
int canOpen(const char *ifname)
{
    int s = ::socket(PF_CAN, SOCK_RAW, CAN_RAW);
    if (s < 0) { std::perror("socket"); return -1; }
    struct ifreq ifr {};
    std::strncpy(ifr.ifr_name, ifname, IFNAMSIZ - 1);
    if (::ioctl(s, SIOCGIFINDEX, &ifr) < 0)
    { std::perror("SIOCGIFINDEX"); ::close(s); return -1; }
    struct sockaddr_can addr {};
    addr.can_family  = AF_CAN;
    addr.can_ifindex = ifr.ifr_ifindex;
    if (::bind(s, (struct sockaddr *)&addr, sizeof addr) < 0)
    { std::perror("bind"); ::close(s); return -1; }
    return s;
}

/// 发一个 5 字节位置帧：ID=id(1~16)，[0x1E][int32 小端 = rad*10000]。
bool sendJointTarget(int s, int id, double joint_rad)
{
    const int32_t v = (int32_t)std::lround(joint_rad * kAngleScale);
```

```
  struct can_frame f {};
  f.can_id  = (canid_t)id;       // 标准帧：11 位，ID 1~16
  f.can_dlc = 5;                 // 必须 5
  f.data[0] = kCmdSetPos;        // 0x1E
  f.data[1] = (uint8_t)( v       & 0xFF);
  f.data[2] = (uint8_t)((v >>  8) & 0xFF);
  f.data[3] = (uint8_t)((v >> 16) & 0xFF);
  f.data[4] = (uint8_t)((v >> 24) & 0xFF);
  return ::write(s, &f, sizeof f) == (ssize_t)sizeof f;
}

/// 收一帧（非阻塞返回 false）。
bool tryRecv(int s, struct can_frame &f)
{
  return ::read(s, &f, sizeof f) == (ssize_t)sizeof f;
}

int main(int argc, char **argv)
{
  const char *ifname = (argc > 1) ? argv[1] : "can4";
  int s = canOpen(ifname);
  if (s < 0) return 1;
  std::printf("[上位机] 已绑定 %s，开始 10Hz 位置流（Ctrl-C 结束）\n",
              ifname);

  long long hb_last_ms = 0;      // 最近一次收到心跳的时刻
  const auto t0 = std::chrono::steady_clock::now();
  for (long long k = 0; ; ++k)
  {
    const double t = k * 0.1;    // 样点时刻(s)，10Hz
    const double amp = 0.6 * std::sin(2 * M_PI * 0.05 * t);

    // —— ① 规划：填 16 个关节的目标角（这里用示例正弦，请替换为你的
    //      RL 策略 / 步态规划输出；单位 rad，ccw 为正）
    double q[kJoints + 1] = {0};  // q[1..16]，q[0] 不用
    q[2] = q[3] = amp;           // 腿1 J2/J3（示例）
    q[6] = q[7] = amp;           // 腿2 J2/J3（示例）

    // —— ② 下发：16 帧一周期
    for (int id = 1; id <= kJoints; ++id)
      sendJointTarget(s, id, q[id]);

    // —— ③ 收反馈 / 心跳（本周期内非阻塞地把可读的都读掉）
    struct can_frame f;
    while (tryRecv(s, f))
    {
      if (f.can_id == kHeartbeatId && f.can_dlc == 1)
      {
        hb_last_ms = std::chrono::duration_cast<
          std::chrono::milliseconds>(
          std::chrono::steady_clock::now() - t0).count();
      }
      else if (f.can_id >= 1 && f.can_id <= kJoints
               && f.can_dlc == 5 && f.data[0] == kCmdGetStatus)
      {
        const int32_t raw = (int32_t)(
          (uint32_t)f.data[1]       |
          ((uint32_t)f.data[2] <<  8) |
          ((uint32_t)f.data[3] << 16) |
          ((uint32_t)f.data[4] << 24));
        const double fb = raw / kAngleScale;  // 实际关节角
        (void)fb;  // TODO: 与 q[] 比较做跟踪监视/安全判断
      }
    }

    // —— ④ 心跳监护：超过 600ms 没心跳 → 下位机失联，进入安全策略
```

```
    const long long now_ms = std::chrono::duration_cast<
      std::chrono::milliseconds>(
      std::chrono::steady_clock::now() - t0).count();
    if (hb_last_ms != 0 && now_ms - hb_last_ms > kHeartbeatMs)
    {
      std::printf("[上位机][警告] 超过 %dms 无心跳，下位机失联！\n",
            kHeartbeatMs);
      break;                // 或走你的急停策略
    }

    // —— ⑤ 按 10Hz 节拍休眠（用绝对时刻对齐，避免累积漂移）
    std::this_thread::sleep_until(
      t0 + std::chrono::milliseconds((long long)((k + 1) * 100)));
  }

  // —— ⑥ 收尾：退出流模式（电机保持当前目标，不抱闸）
  struct can_frame stop {};
  stop.can_id  = 1;
  stop.can_dlc = 1;
  stop.data[0] = kCmdStop;
  (void)!::write(s, &stop, sizeof stop);  // 忽略返回值以消 -Wunused-result
  ::close(s);
  return 0;
}
```

7.4 RK3588 侧部署：接口从哪来、怎么命名、怎么编译

上位机是 **RK3588** 板卡（4×Cortex-A76 + 4×Cortex-A55，aarch64）。它**没有原生 CAN 控制器可用**到本项目的接线上，实际做法是外挂一路 USB-CAN，由内核把它暴露成标准的 `canX` SocketCAN 接口——因此第 7.1~7.3 节的代码在 RK3588 上可以**原样**运行，不需要任何驱动 SDK。

7.4.1 第一步：先让 RK3588 上出现一个 UP 的 can 接口

板子出厂时内核一般已带 `can`、`can_raw`、`vcan` 模块，大概率已内置。按实际外挂的 USB-CAN 走下面三选一：

外挂方案	内核依赖	接口名	备注
鲲弘 kcan（与下位机同款）	厂商 SDK 的 netdev 驱动	`canX`	需在 RK3588 上装 SDKTools 并 `make netdev`，与下位机同一套流程
8devices / PEAK / 广成等	`can` + `can_raw` + 厂商 `usb` 驱动	`canX`	多数已进主线内核，插上即出现
CAN 转串口 / SPI 模块	厂商驱动	`canX` 或 `slcan0`	若为 slcan，需 `slcand` 先建接口，仍可用同一份代码

确认接口出现、并把它拉起来（1 Mbps，与下位机总线必须一致）：

```
ip -br link show type can       # 看有哪些 canX（DOWN 状态正常）
sudo ip link set can4 type can bitrate 1000000
sudo ip link set can4 up
ip -details link show can4      # 必须看到 state ERROR-ACTIVE
```

关键判据：`ip -details` 里 `can state ERROR-ACTIVE` 且 `berr-counter tx 0 rx 0` 才算物理层健康。若显示 `ERROR-PASSIVE` 且 `tx` 计数快速上涨，说明**线没接好或对端没上电**（悬空通道），此时发帧会一路报错并最终把发送缓冲区塞满（`ENOBUFS`）——不要在这种状态下做压力测试。

7.4.2 第二步：接口命名不要依赖“一定是 can4”

RK3588 上 `canX` 的编号取决于**插入顺序**和内核枚举顺序，同一块板子换 USB 口或换了适配器就可能变成 `can0`。两个稳妥做法，任选其一：

```
方法 A（推荐，零配置）：程序只认命令行参数，启动时人眼确认后传入
  ./host_stream can0     # 先用 ip -br link 看清实际名字
```

```
方法 B（固定命名）：按 USB 序列号/U盘口生成 udev 规则，绑定成固定名
  # /etc/udev/rules.d/80-taihu-host-can.rules
  ACTION=="add", SUBSYSTEM=="net", ATTRS{serial}=="<你的适配器序列号>",
    NAME="hostcan0"
  # 生效后程序里传 hostcan0 即可
```

千万不要把上位机接在下位机同一块鲲弘适配器的 can0~can3 上：那 4 条是**电机总线**，下位机会把上位机发来的 ID 1~4 当成“腿 1 的关节
1~4”再转给真电机，导致上位机指令被真机执行。上位机必须占用**独立**的一路接口（下位机上对应 can4 / can5）。

7.4.3 第三步：编译与自检顺序

```
# 在 RK3588 上本机编译（推荐，无需交叉工具链）
g++ -O2 -std=c++17 -Wall -Wextra -o host_stream host_stream.cpp

# 若在 x86 开发机上交叉编译（Ubuntu 需先装 g++-aarch64-linux-gnu）
aarch64-linux-gnu-g++ -O2 -std=c++17 -static \
  -o host_stream_aarch64 host_stream.cpp

# 上板后按这个顺序自检，每步都通过再进下一步
ip -br link show type can        # ① 接口存在
ip -details link show can0 | grep 'can state'   # ② ERROR-ACTIVE
./host_stream can0 --dry-run      # ③ 只跑节拍不发帧（如有该开关）
candump -td can0                  # ④ 另开终端看是否出现 0x161 心跳
```

静态链接（`-static`）可避免 RK3588 上 glibc 版本与开发机不一致导致的 `GLIBC_2.xx not found`；若板子上本来就编，直接用本机 `g++` 最省事。另注意：**不要**用 `-mcpu=native` 在开发机上交叉编译——那会生成 A76 专属指令，在 A55 核上可能非法。

7.5 逐项对照检查表（出错时按此顺序排查）

#	检查项	正确做法	常见错误
1	接口名	上位机（RK3588）接在**独立**接口；编号以 `ip -br link` 实际为准，不能假设一定是 can4	与电机总线 can0~can3 共用 → 真电机直接执行 ID 1~4
2	帧类型	标准帧（11 位 ID）	发成扩展帧 → 下位机直接丢弃
3	DLC	位置帧固定 5	DLC=8 补零 / DLC=4 → 不执行
4	data[0]	0x1E（位置）/ 0x02（退流）/ 0x41（查询）	写成 0x1E 以外的功能码 → 不动作
5	字节序	小端：最低字节放 data[1]（aarch64 同为小端）	大端拼装 → 角度离谱或越限被拒
6	定点比例	rad × 10000（不是 ×1000，不是 deg）	用度或 ×1000 → 角度差一个量级
7	符号	ccw（逆时针）= 正；负值用 int32 补码	绝对值+符号位 → 反向
8	帧 ID 映射	ID = (腿−1)×4 + 关节号	腿号关节号顺序颠倒
9	周期	10Hz（每 100ms 一组 16 帧）	过快无益；过慢 <1Hz 会触发断流
10	使能	下位机启动时已自动解抱闸，无需上位机发	误以为要发「使能帧」；或电机抱闸却反复重发位置帧
11	限位	关节端 J1/J4 ±90°、J2 ±120°、J3 −30°~+180°	越限帧被**静默丢弃**（下位机仅首次打印告警）
12	收尾	结束时发一次 0x02	直接断开 → 下位机等 1s 断流超时后才退出
13	波特率	RK3588 侧与下位机侧都设 1000000，且两端各并 120Ω	一端 500k 一端 1M → 全是错误帧，收不到心跳
14	架构差异	aarch64/小端 与本协议完全兼容，无需做任何字节序转换	误加 `htole32` 之外的二次翻转 → 数值错乱

最有效的单点验证：先只发一帧（见 7.1 的 cansend 示例），确认该关节按预期角度运动，再去跑 16 帧全量轨迹。若单帧正确而全量不动，问题一定在使能或限位上，不在帧格式上。

8. 上位机接口 API（最小封装参考）

若已有自己的 CAN 封装，只需补这三个函数；语义与第 7 章完全一致。帧结构与钛虎电机写命令帧同构，可与控制真电机的代码共用 can_send：

```
/* ---------- 单关节目标发送（10Hz 周期内循环 16 次） ---------- */
/* id：1~16（=(leg-1)*4+joint，leg/joint 均 1 起）
 * joint_rad：关节端目标角（rad，ccw 为正）。
 * 帧：标准帧 DLC=5，[0x1E][int32 小端 = rad*10000]。 */
int32_t host_send_joint_target(can_handle_t h, int id, double joint_rad) {
  uint8_t d[5];
  int32_t v = (int32_t)lround(joint_rad * 10000.0);
  d[0] = 0x1E;
  d[1] = (uint8_t)( v       & 0xFF);  /* 小端 */
  d[2] = (uint8_t)((v >>  8) & 0xFF);
  d[3] = (uint8_t)((v >> 16) & 0xFF);
  d[4] = (uint8_t)((v >> 24) & 0xFF);
  return can_send(h, (uint32_t)id, 5, d);  /* 标准帧 ID=id */
}

/* ---------- 退出流模式（轨迹回放结束时调用） ---------- */
int32_t host_stream_stop(can_handle_t h) {
  uint8_t d[1] = {0x02};
  return can_send(h, 1, 1, d);        /* 任意关节 ID 均可 */
}

/* ---------- 状态请求（可选；下位机本就 10Hz 主动反馈） ---------- */
int32_t host_query_joint(can_handle_t h, int id) {
  uint8_t d[1] = {0x41};
  return can_send(h, (uint32_t)id, 1, d);
}
/* 收反馈帧：ID∈[1,16], DLC=5, data[0]==0x41：
 *  joint_rad = (int32)(data[1] | data[2]<<8 | data[3]<<16
 *                | data[4]<<24) / 10000.0;
 * 收心跳帧：ID==0x161, DLC=1：
 *  stream_armed = data[0] & 0x01;  stream_online = data[0] & 0x02; */
```

上位机主循环调用流程：

```
can_init(1000000);                  /* 1. 初始化 CAN 卡   */
for (t = 0; ; t += 0.1) {           /* 2. 10Hz 步进       */
  plan_gait(t, joint_rad);          /*   RL策略/步态规划  */
  for (id = 1; id <= 16; id++)      /*   逐关节发目标帧   */
    host_send_joint_target(can, id, joint_rad[id]);
  if (heartbeat_timeout_600ms()) {  /* 3. 心跳监护        */
    emergency(); return;            /*   失联→安全策略    */
  }
}
host_stream_stop(can);              /* 4. 结束：退出流模式 */
```

9. 链路自测：不接电机先验通路

上位机与下位机的 CAN 通讯可以**完全不接任何电机**先验证通过——用虚拟 CAN 接口（vcan）在进程间回环，是调试帧格式、时序、使能逻辑最快的手段。注意：下位机的**电机总线**（腿 1~4）接口名硬编码为 can0~can3，无法改用虚拟接口；因此自测时**上位机链路走 vcan0**，电机总线仍指向 can0~can3（未接电机时该腿的读操作会自动退避，不影响上位机链路自测）。

9.1 方案一：本机双终端（vcan0 回环，零接线）

```
# —— 终端 A（一次配置，需 root；vcan 创建的接口重启即消失）
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan   # 已存在则报 File exists，忽略
```

```
sudo ip link set up vcan0
ip -br link show vcan0              # 期望看到 UNKNOWN 状态（正常）

# —— 终端 A：跑下位机，上位机接口填 vcan0
./build/walk_control_V1
#   腿号：至少 1 条（如 1；注意该腿会去 open can0）
#   上位机接口：vcan0

# —— 终端 B：跑上位机（Python 示例或自己写的 C++ 程序）
python3 tools/up_walk_control_V1.py --can vcan0 --only-mapped
#   或：./host_stream vcan0

# —— 终端 C：旁路观察（强烈建议开）
candump vcan0          # 看双向帧；data[0]=1E 是下发，41 是反馈
candump -td vcan0      # 带时间戳，核查 10Hz 节拍
candump vcan0,161:7FF    # 只看心跳（0x161 帧）
```

vcan 的帧**只在本机进程间回环**，不经过任何硬件，因此不需要终端电阻、不需要真实 CAN 卡、波特率参数被忽略。这是验证「上位机帧写对没有 / 下位机拆帧对没有」的干净环境。

注意：`--only-mapped` 只发有数据的关节（4 帧/周期）；不加该参数则发全 16 帧。若统计到的帧数是 3233（≈201×16）说明flag 没生效，应为 821（≈201×4）。上位机脚本打印的进度行显示的是**下位机反馈的实际角**，不是发送值——想看发送值请用 candump。

9.2 方案二：真实硬件双接口（can4 ↔ can5）

若必须验证真实 CAN 卡与线束：把上位机 CAN 卡的 CAN_H/CAN_L 接到下位机**另一个空闲**接口（can5），**两端各并一只 120Ω**，两个接口都 up 到 1Mbps。

```
sudo ./build/kcanctl up can4 1000000   # 上位机侧
sudo ./build/kcanctl up can5 1000000   # 下位机侧
# 健康判定（关键）：两端都 up 后，ACR/ACK 应能互相确认
ip -d link show can4 | grep "can state"
#   can state ERROR-ACTIVE (berr-counter tx 0 rx 0)  ← 正常
#   can state ERROR-PASSIVE (berr-counter tx 128)    ← 悬空！缺 120Ω 或无对端
```

⚠ **不要在悬空（无对端 ACK）的通道上跑真实负载**：实测 10Hz×16帧双向负载约 20s 即会把接口卡死，报 `No buffer space available`，且需 down/up 才能恢复。先确认 `berr-counter tx 0` 再发流量。

⚠ 悬空总线上的 `candump` 静默**不能**证明电机离线——钛虎电机不主动发帧，只在被写/被读时回复。判断电机在线请用下位机启动时的探测行（`电机在线性：J1 J2 J3 J4`，× 表示无响应）。

9.3 三步验收流程

步骤	操作	通过标准
① 帧能到	下位机启动 → 上位机发一帧 → 下位机终端打印 `[流] 收到首帧 0x1E（ID 2 = 腿1 J2）：进入流模式`	打印出现，且 ID 与腿/关节对应正确
② 值对得上	candump 对照 §7.2.1 的字节表逐字节比对 data[1..4]	字节序、定点比例、符号全部一致
③ 会动且跟随	接真电机，发一条 0→+70°→0 的缓慢轨迹，同时看 0x41 反馈是否追随目标	关节按预期方向运动；反馈角与目标偏差 < 数度；若完全不动 → 检查是否使能（§3.5）
④ 收尾干净	轨迹结束发 0x02，或直接停发	下位机打印 `[流] 收到 0x02：退出流模式`，或 1s 后自动退出；电机保持最后姿态不塌

10. 下位机（本机）使用说明

```
# 1. 启动 CAN 接口（1Mbps）：4 条电机总线 + 1 条上位机总线
sudo ./build/kcanctl up can0 1000000   # can1~can3 同理
sudo ./build/kcanctl up can4 1000000   # 上位机链路（独立接口）
```

```
# 2. 编译并启动下位机程序
cmake --build build --target walk_control_V1
./build/walk_control_V1
#  → 选择启用腿号（直接回车 = 全部 4 腿）
#  → 输入上位机 CAN 接口名（如 can4；直接回车 = 跳过上位机链路，
#    仅键盘命令可用；输入 can0~can3 会被拒绝——不能与电机总线共用）
#  → 启动时自动使能（解抱闸），终端打印：
#    [腿1][使能] 抱闸已解除（4/4 电机应答），当前姿态已作为保持目标

# 3. 运行期键盘命令
pos [腿1-4]    # 查询关节实际位置（读 10Hz 缓存，零总线流量）
move/goto ...  # 五次多项式轨迹命令（流模式中暂缓）
stat         # 流模式统计（各腿收帧/丢弃/电机在线/使能）
arm          # 使能：解除抱闸（启动时已自动执行；急停后重新使能）
disarm       # 去使能：全部电机抱闸并锁定
estop        # 急停：全部电机抱闸 + 退出流模式 + 锁定
resume       # 解除急停锁定并重新使能（解抱闸）
quit         # 退出：电机抱闸后关闭总线
```

下位机收到首帧 0x1E 后自动进入流模式（终端打印 `[流] 收到首帧 0x1E：进入流模式`）；1s 断流或收到 0x02 自动退出。流模式期间 move/goto 命令暂缓，pos/estop 始终可用；estop 后进入锁定状态，上位机 0x1E 帧被丢弃，直到 resume / arm。

测试建议：先用 `candump can4` 观察上位机帧是否到达本机（应看到 ID 001~010 的 5 字节帧与下位机回的 0x161 心跳）；下位机终端 `stat` 命令核对各腿收帧率应为 ≈40 帧/s（4 关节 × 10Hz）；再 `candump can0` 确认电机总线上出现 0x1E 写帧（ID 1~4）。